

Comparison of LoRA and Knowledge Editing for Improving Neural Decompilation

Bachelor's Thesis Presentation

Hendrik Lohmar

Supervisor: Prof. Dr. Artur Andrzejak, Akin Yilmaz

Heidelberg University
Faculty of Mathematics and Computer Science
Institute of Computer Science
Artificial Intelligence for Programming (AIP)

February 2, 2026

Outline

The Decompilation Problem	2
Methodology	8
Results	17
Error Analysis	23
Addressing Errors	26
Discussion	29
Takeaways	34
Bibliography	45

The Decompilation Problem

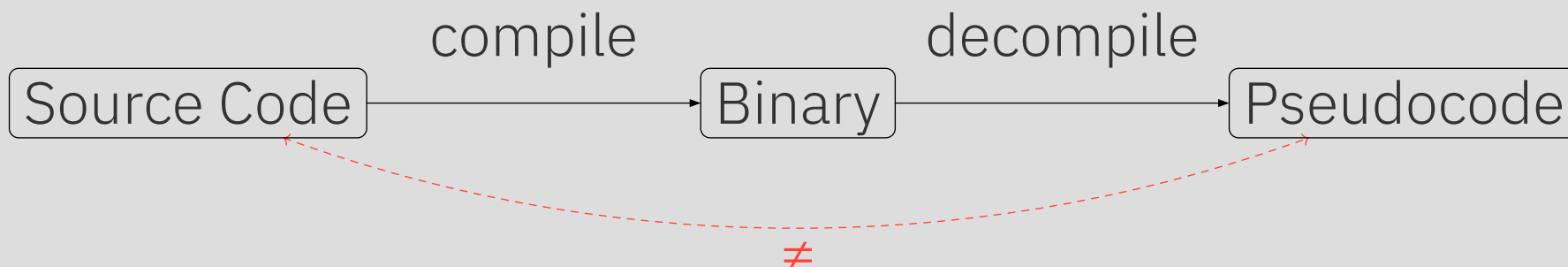
Why Decompile Matters

- Security analysis & vulnerability research
- Malware detection
- Legacy software maintenance
- Reverse engineering

The Decompile Problem

Compilation is *lossy*; information is discarded:

- Variable names → registers/stack offsets
- Type information → memory sizes
- Comments & formatting → gone



Ghidra Decompiler Output

Original C code:

```
int sum_array(int *arr, int n) {  
    int total = 0;  
    for (int i = 0; i < n; i++) {  
        total += arr[i];  
    }  
    return total;  
}
```

Ghidra pseudocode:

```
undefined8 sum_array(long param_1, int param_2) {  
    undefined8 uVar1;  
    int local_c;  
    int local_8;  
    local_c = 0;  
    local_8 = 0;  
    while (local_8 < param_2) {  
        local_c = local_c +  
            *(int *) (param_1 +  
                (long) local_8 * 4);  
        local_8 = local_8 + 1;  
    }  
    uVar1 = CONCAT44(local_c, local_c);  
    return uVar1;  
}
```

Ghidra Artifacts

Artifact	Description	Example
Type annotations	Size-based types	undefined4 → int
Variable names	Stack offsets	local_1c, param_1
Pointer arithmetic	Explicit address calculations	*(int*)(ptr + i*4)
Data references	Address-based labels	DAT_00104000, _LC0

Goal: Transform pseudocode into valid, compilable C code

Research Questions

- RQ1. How effectively does **LoRA** improve decompilation correctness?
- RQ2. Can **Knowledge Editing** correct systematic decompiler artifacts?
- RQ3. What error types are **addressable**, and what **limitations** remain?
- RQ4. How does **error-specific** training compare to **general** fine-tuning?

Methodology

CodeLlama

- Based on Llama 2, trained on 500B code tokens
- 7B to 34B parameters
- Strong at code understanding & generation
- Instruction-tuned variant follows directives

Choice: CodeLlama-7B-Instruct – practical size for fine-tuning, instruction-following capability, openly available

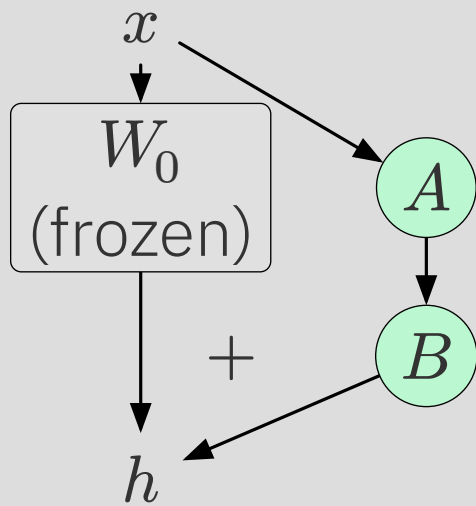
Prompt Template

```
<s>[INST] You are an expert C decompiler.  
Refine the following Ghidra pseudocode into  
valid, compilable C code.  
...  
Pseudocode:  
{ghidra_pseudocode}  
[/INST]
```

Low-Rank Adaptation (LoRA)

Challenge: Fine-tuning LLMs requires updating billions of parameters

Solution: Inject small trainable matrices, keep base model frozen



- Train small matrices A , B
- ~2% of parameters
- No inference latency
- Compatible with QLoRA (4-bit)

Knowledge Editing (ROME)

Hypothesis: Decompilation errors = incorrect facts → surgically correct them

ROME: Rank-one updates to MLP weights

Original use: “*Eiffel Tower in Paris*” → “*...London*”

Decompilation as knowledge?

```
(undefined4, "corresponds to C  
type", int)
```

Experimental Pipeline



Training Data: ExeBench

- ~4,000 C functions
- GCC → Ghidra
- Filtered for Quality

Evaluation: HumanEval-Decompile

- 151 functions with test harnesses
- 5 runs per config

Training Configuration

Base model	CodeLlama-7B-Instruct
Quantization	4-bit
LoRA targets	All attn + MLP projections
Epochs	3
Learning rate	2×10^{-4}
Batch size	16

Hardware: Google Colab A100, bwHPC H100

Evaluation Metrics

- **Pass@1:** Compile → Execute → All assertions pass
- **Compile Rate:** Syntactically valid output

5 runs per config (temperature 0.2, top-p 0.95)

Experimental Conditions

Condition	Description
Baseline	Unmodified CodeLlama-7B-Instruct
General LoRA	Fine-tuned on ~4,000 ExeBench samples
ROME-edited	Knowledge Editing
Targeted LoRA	Trained only on synthetic error examples
Mixed LoRA	General data + synthetic error examples (2:1 ratio)

Results

LoRA Fine-Tuning Results

Model	Pass@1	Compile
Baseline	15.50%	18.94%
LoRA (r=64)	23.95%	84.33%

- Pass@1: +54% relative
- Compile: **4.5× increase**

Baseline: Outputs
undefined4, CONCAT44, etc.

LoRA: Learns to remove
these artifacts

Knowledge Editing: A Negative Result

Model	Pass@1	Compile
Baseline	15.50%	18.94%
ROME-edited	15.23%	22.19%

Why doesn't it work? The model **already knows** the facts:

Q: “*What C type is undefined4?*” → A: “*int*”

The problem is **inconsistent application** of knowledge, not missing knowledge.

Qualitative Example: Ghidra Input

```
undefined8 func0(int *param_1,uint param_2)
{
    int *piVar1;

    if (0 < (int)param_2) {
        piVar1 = param_1 + (int)param_2;
        if ((param_2 & 1) != 0) {
            *param_1 = *param_1 + 1;
            param_1 = param_1 + 1;
            if (param_1 == piVar1) {
                return 0;
            }
        }
        do {
            *param_1 = *param_1 + 1;
            param_1[1] = param_1[1] + 1;
            param_1 = param_1 + 2;
        } while (param_1 != piVar1);
    }
    return 0;
}
```

Artifacts:

- undefined8 return type
- uint parameter type
- Unrolled loop structure

Original intent: Increment each element of an array

Qualitative Example: Model Outputs

Baseline (FAIL):

```
int func0(int *param_1, uint param_2)
{
    int *piVar1 = param_1 + (int)param_2;
    if (param_2 & 1) {
        *param_1 = *param_1 + 1;
        param_1 = param_1 + 1;
        if (param_1 == piVar1)
            return 0;
    }
    do {
        *param_1 = *param_1 + 1;
        param_1[1] = param_1[1] + 1;
        param_1 = param_1 + 2;
    } while (param_1 != piVar1);
    return 0;
}
```

Preserves unrolled structure

LoRA (PASS):

```
void func0(int *p, int n)
{
    int i;
    for (i = 0; i < n; i++)
        *p++ += 1;
}
```

Recognizes intent, simplifies
loop

Result Stability Analysis

Category	Count	%	Implication
Consistently passing	20	13.2%	Robust solutions
Consistently failing	99	65.6%	Fundamental limitations
Flaky (varies)	32	21.2%	Near decision boundary

Key insight: 21% of samples vary with minor sampling changes → single-run evaluation is misleading; multi-run evaluation essential

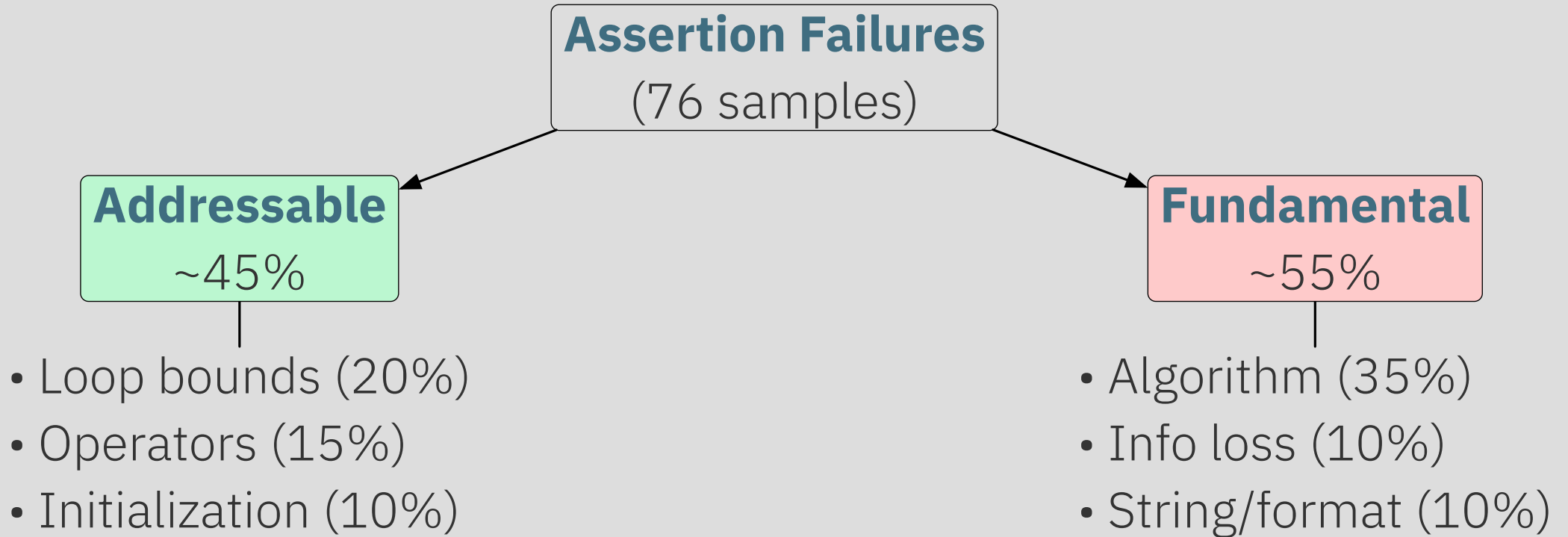
Error Analysis

Failure Types

Failure Type	%	Meaning
Assertion failure	76.8%	Compiles but wrong output
Compilation error	22.2%	Invalid syntax or types
Timeout	1.0%	Infinite loop

Key finding: Most failures are semantic – syntactic correction $\neq$ semantic correctness

Semantic Error Taxonomy



Addressing Errors

Error-Specific Fine-Tuning

Approach: Synthetic data with systematic errors

- Loop bounds: $i < n \rightarrow i \leq n$
- Operators: $< \rightarrow >$
- Initialization: remove/corrupt

Model	Pass@1	Compile	Training Data
Baseline	15.50%	18.94%	None
Targeted LoRA	20.13%	21.06%	Only error examples
General LoRA	23.95%	84.33%	ExeBench samples
Mixed LoRA	28.08%	64.50%	General + errors (2:1)

Summary of All Results

Model	Pass@1	Compile	Δ Pass@1
Baseline	15.50%	18.94%	—
ROME-edited	15.23%	22.19%	−0.27
Targeted LoRA	20.13%	21.06%	+4.63
General LoRA	23.95%	84.33%	+8.45
Mixed LoRA	28.08%	64.50%	+12.58

Best functional correctness: **81% relative improvement** over baseline

Discussion

The Syntactic-Semantic Gap

Syntactic adaptation: Readily achievable, even $r=8$ LoRA reaches 82% compile rate

Semantic correctness: Diminishing returns, all general configs ~23-24% Pass@1

Why? ~55% of errors are fundamental (info loss, ambiguity)

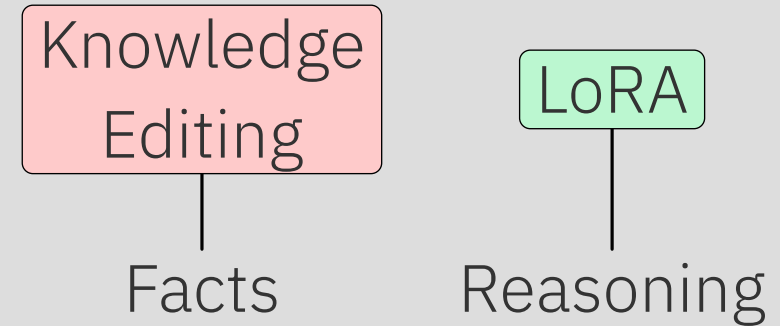
Why Knowledge Editing Fails

Knowledge Editing:

- Atomic facts
- Single corrections

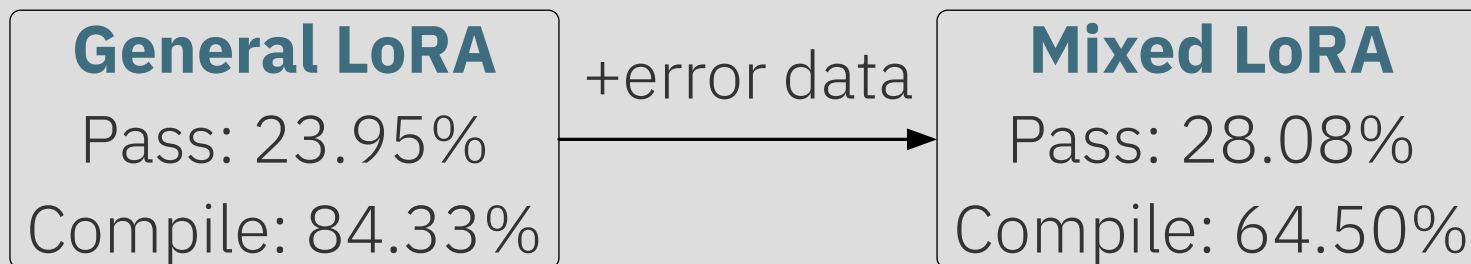
Decompilation:

- Context-dependent
- Multi-step reasoning



Decompilation is **reasoning**, not retrieval.

Trade-offs in Mixed Training



Observation: Improving semantic correctness trades off against syntactic validity

Future direction: Multi-objective optimization, curriculum learning

Comparison with State-of-the-Art

System	Method	Training Data	Performance
LLM4Decompile	Full fine-tune	Billions of tokens	36.71%
This work	LoRA	~4,000 samples	28.08%

- Achieve **76% of SOTA** with orders of magnitude less data
- Parameter-efficient: only ~2% of weights trained
- Practical for resource-constrained settings

Takeaways

Summary of Contributions

1. **LoRA works:** 4.5× compile rate, 54-81% relative Pass@1 improvement
2. **Knowledge Editing doesn't:** decompilation is reasoning, not retrieval
3. **Error taxonomy:** 45% addressable, 55% fundamental
4. **Syntactic-semantic gap:** compile rate $\neq$ correctness

Future Work

- **Training:** Curriculum learning, multi-objective optimization, RLHF
- **Scaling:** Larger models, more training data
- **Better inputs:** Preserve more semantic info from decompiler
- **Architecture:** Chain-of-thought, RAG, multi-pass refinement
- **Extensions:** Other compilers, optimization levels, decompilers

5 Minute Demo

Any questions or thoughts?

Backup Slides

LoRA: Mathematical Details

$$h = W_0x + \Delta Wx = W_0x + \left(\frac{\alpha}{r}\right) \cdot BAx$$

$W_0 \in \mathbb{R}^{d \times k}$	Frozen pretrained weights
$A \in \mathbb{R}^{r \times k}$	Trainable down-projection
$B \in \mathbb{R}^{d \times r}$	Trainable up-projection
$r \ll \min(d, k)$	Low rank (e.g., 64)
α	Scaling factor

Why it works: Weight updates have low intrinsic rank, full-rank updates aren't needed.

LoRA Hyperparameter Search

Rank / Alpha	Pass@1 (%)	Compile (%)
8 / 8	20.93 $\pm$ 0.79	82.38 $\pm$ 0.32
16 / 16	21.46 $\pm$ 2.03	82.91 $\pm$ 1.59
32 / 32	23.44 $\pm$ 1.60	84.77 $\pm$ 0.59
32 / 64	23.71 $\pm$ 1.84	83.97 $\pm$ 0.77
64 / 64	23.95 $\pm$ 1.35	84.33 $\pm$ 1.19
64 / 128	22.78 $\pm$ 0.90	85.56 $\pm$ 1.59
128 / 128	23.18 $\pm$ 2.09	84.50 $\pm$ 0.99

Even $r=8$ achieves 82% compile; Pass@1 plateaus at $r=32-64$; diminishing returns beyond.

QLoRA: Quantized LoRA

Problem: 7B model needs ~28GB in FP32, ~14GB in FP16

Solution: 4-bit NormalFloat (NF4) quantization + LoRA

How it works:

- Base weights quantized to 4-bit
- LoRA adapters in FP16, receive gradients
- Dequantize on-the-fly for forward pass

Benefits:

Memory	~6GB for 7B model
Speed	Minimal overhead
Quality	Near FP16 performance

ROME: Mathematical Details

Goal: Edit $(s, r, o) \rightarrow (s, r, o^*)$ by modifying MLP weights

$$W^* = W + \Delta, \quad \Delta = (C^{-1}k_*)(v_* - Wk_*)^\top$$

k_*	Key vector: hidden state at subject s
v_*	Target value: encodes new object o^*
C	Covariance of key vectors
W	Original MLP weights

Intuition: Insert new key-value pair into MLP's associative memory

Failure Example: Loop Bound Error

Model output:

```
int sum(int *arr, int n) {  
    int total = 0;  
    for (int i = 0; i <= n; i++) {  
        total += arr[i];  
    }  
    return total;  
}
```

`i <= n` causes out-of-bounds access

Expected:

```
int sum(int *arr, int n) {  
    int total = 0;  
    for (int i = 0; i < n; i++) {  
        total += arr[i];  
    }  
    return total;  
}
```

`i < n` is correct bound

Category: Addressable, off-by-one errors are systematic and trainable

Failure Example: Algorithm Misidentification

Model output:

```
int find_max(int *arr, int n) {  
    int max = 0;  
    for (int i = 0; i < n; i++) {  
        if (arr[i] > max)  
            max = arr[i];  
    }  
    return max;  
}
```

Fails for all-negative arrays

Expected:

```
int find_max(int *arr, int n) {  
    int max = arr[0];  
    for (int i = 1; i < n; i++) {  
        if (arr[i] > max)  
            max = arr[i];  
    }  
    return max;  
}
```

Initialize with first element

Category: Fundamental, requires understanding algorithm intent from ambiguous pseudocode

Bibliography

- [1] A. V. Aho, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, Reprinted, with corr., [36. Druck]. in Addison-Wesley Series in Computer Science. Reading, Mass.: Addison-Wesley, 2002.
- [2] J. Armengol-Estapé, J. Woodruff, A. Brauckmann, J. W. D. S. Magalhães, and M. F. P. O'Boyle, “ExeBench: An ML-scale Dataset of Executable C Functions,” in *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*, San Diego CA USA: ACM, June 2022, pp. 50–59. doi: 10.1145/3520312.3534867.
- [3] Baden-Württemberg Ministerium für Wissenschaft, Forschung und Kunst, “bwHPC.” [Online]. Available: <https://www.bwhpc.de/>
- [4] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer Normalization,” no. arXiv:1607.06450. arXiv, July 2016. doi: 10.48550/arXiv.1607.06450.
- [5] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, “Curriculum Learning,” in *Proceedings of the 26th Annual International Conference on Machine Learning*, Montreal Quebec Canada: ACM, June 2009, pp. 41–48. doi: 10.1145/1553374.1553380.
- [6] M. Chen *et al.*, “Evaluating Large Language Models Trained on Code,” no. arXiv:2107.03374. arXiv, July 2021. doi: 10.48550/arXiv.2107.03374.
- [7] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient Finetuning of Quantized LLMs,” no. arXiv:2305.14314. arXiv, May 2023. doi: 10.48550/arXiv.2305.14314.
- [8] C. Fu *et al.*, “A Neural-based Program Decompiler,” no. arXiv:1906.12029. arXiv, June 2019. doi: 10.48550/arXiv.1906.12029.

Bibliography

- [9] M. Geva, R. Schuster, J. Berant, and O. Levy, “Transformer Feed-Forward Layers Are Key-Value Memories,” in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, Online and Punta Cana, Dominican Republic: Association for Computational Linguistics, 2021, pp. 5484–5495. doi: 10.18653/v1/2021.emnlp-main.446.
- [10] GNU Project, “GNU Compiler Collection.” [Online]. Available: <https://gcc.gnu.org/>
- [11] Google Research, “Google Colaboratory.” [Online]. Available: <https://colab.research.google.com/>
- [12] D. Han, M. Han, and Unsloth team, “Unsloth.” [Online]. Available: <https://github.com/unslothai/unsloth>
- [13] T. Hartvigsen, S. Sankaranarayanan, H. Palangi, Y. Kim, and M. Ghassemi, “Aging with GRACE: Lifelong Model Editing with Discrete Key-Value Adaptors,” no. arXiv:2211.11031. arXiv, Oct. 2023. doi: 10.48550/arXiv.2211.11031.
- [14] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” no. arXiv:1512.03385. arXiv, Dec. 2015. doi: 10.48550/arXiv.1512.03385.
- [15] Hex-Rays, “IDA Pro.” [Online]. Available: <https://hex-rays.com/ida-pro>
- [16] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997, doi: 10.1162/neco.1997.9.8.1735.
- [17] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, “The Curious Case of Neural Text Degeneration,” no. arXiv:1904.09751. arXiv, Feb. 2020. doi: 10.48550/arXiv.1904.09751.
- [18] N. Houlsby *et al.*, “Parameter-Efficient Transfer Learning for NLP,” no. arXiv:1902.00751. arXiv, June 2019. doi: 10.48550/arXiv.1902.00751.

Bibliography

- [19] Z. Huang, Y. Shen, X. Zhang, J. Zhou, W. Rong, and Z. Xiong, “Transformer-Patcher: One Mistake Worth One Neuron,” no. arXiv:2301.09785. arXiv, Jan. 2023. doi: 10.48550/arXiv.2301.09785.
- [20] P. Hu, R. Liang, and K. Chen, “DeGPT: Optimizing Decompiler Output with LLM,” in *Proceedings 2024 Network and Distributed System Security Symposium*, San Diego, CA, USA: Internet Society, 2024. doi: 10.14722/ndss.2024.24401.
- [21] E. J. Hu *et al.*, “LoRA: Low-Rank Adaptation of Large Language Models,” no. arXiv:2106.09685. arXiv, Oct. 2021. doi: 10.48550/arXiv.2106.09685.
- [22] Y. Jiang *et al.*, “Learning to Edit: Aligning LLMs with Knowledge Editing,” no. arXiv:2402.11905. arXiv, June 2024. doi: 10.48550/arXiv.2402.11905.
- [23] N. Jiang *et al.*, “Nova: Generative Language Models for Assembly Code with Hierarchical Attention and Contrastive Learning,” no. arXiv:2311.13721. arXiv, Nov. 2025. doi: 10.48550/arXiv.2311.13721.
- [24] J. Kaplan *et al.*, “Scaling Laws for Neural Language Models,” no. arXiv:2001.08361. arXiv, Jan. 2020. doi: 10.48550/arXiv.2001.08361.
- [25] J. Kirkpatrick *et al.*, “Overcoming Catastrophic Forgetting in Neural Networks,” *Proceedings of the National Academy of Sciences*, vol. 114, no. 13, pp. 3521–3526, Mar. 2017, doi: 10.1073/pnas.1611835114.
- [26] B. Lester, R. Al-Rfou, and N. Constant, “The Power of Scale for Parameter-Efficient Prompt Tuning,” no. arXiv:2104.08691. arXiv, Sept. 2021. doi: 10.48550/arXiv.2104.08691.
- [27] P. Lewis *et al.*, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” no. arXiv:2005.11401. arXiv, Apr. 2021. doi: 10.48550/arXiv.2005.11401.

Bibliography

- [28] X. Li *et al.*, “Model Editing for LLMs4Code: How Far Are We?,” no. arXiv:2411.06638. arXiv, Dec. 2024. doi: 10.48550/arXiv.2411.06638.
- [29] X. Li, S. Li, S. Song, J. Yang, J. Ma, and J. Yu, “PMET: Precise Model Editing in a Transformer,” no. arXiv:2308.08742. arXiv, Mar. 2024. doi: 10.48550/arXiv.2308.08742.
- [30] X. L. Li and P. Liang, “Prefix-Tuning: Optimizing Continuous Prompts for Generation,” no. arXiv:2101.00190. arXiv, Jan. 2021. doi: 10.48550/arXiv.2101.00190.
- [31] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” no. arXiv:1711.05101. arXiv, Jan. 2019. doi: 10.48550/arXiv.1711.05101.
- [32] K. Meng, D. Bau, A. Andonian, and Y. Belinkov, “Locating and Editing Factual Associations in GPT,” no. arXiv:2202.05262. arXiv, Jan. 2023. doi: 10.48550/arXiv.2202.05262.
- [33] K. Meng, A. S. Sharma, A. Andonian, Y. Belinkov, and D. Bau, “Mass-Editing Memory in a Transformer,” no. arXiv:2210.07229. arXiv, Aug. 2023. doi: 10.48550/arXiv.2210.07229.
- [34] E. Mitchell, C. Lin, A. Bosselut, C. Finn, and C. D. Manning, “Fast Model Editing at Scale,” no. arXiv:2110.11309. arXiv, June 2022. doi: 10.48550/arXiv.2110.11309.
- [35] E. Mitchell, C. Lin, A. Bosselut, C. D. Manning, and C. Finn, “Memory-Based Model Editing at Scale,” no. arXiv:2206.06520. arXiv, June 2022. doi: 10.48550/arXiv.2206.06520.
- [36] National Security Agency, “Ghidra.” [Online]. Available: <https://www.nsa.gov/ghidra>

Bibliography

- [37] H. Pan *et al.*, “Precise Localization of Memories: A Fine-grained Neuron-level Knowledge Editing Technique for LLMs,” no. arXiv:2503.01090. arXiv, Mar. 2025. doi: 10.48550/arXiv.2503.01090.
- [38] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, “BLEU: A Method for Automatic Evaluation of Machine Translation,” in *Proceedings of the 40th Annual Meeting on Association for Computational Linguistics - ACL '02*, Philadelphia, Pennsylvania: Association for Computational Linguistics, 2001, p. 311. doi: 10.3115/1073083.1073135.
- [39] A. Paszke *et al.*, “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” no. arXiv:1912.01703. arXiv, Dec. 2019. doi: 10.48550/arXiv.1912.01703.
- [40] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, “Improving Language Understanding by Generative Pre-Training.” 2018.
- [41] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language Models Are Unsupervised Multitask Learners.” 2019.
- [42] B. Rozière *et al.*, “Code Llama: Open Foundation Models for Code.” arXiv, 2023. doi: 10.48550/ARXIV.2308.12950.
- [43] R. Sennrich, B. Haddow, and A. Birch, “Neural Machine Translation of Rare Words with Subword Units,” no. arXiv:1508.07909. arXiv, June 2016. doi: 10.48550/arXiv.1508.07909.
- [44] N. Shazeer, “GLU Variants Improve Transformer,” no. arXiv:2002.05202. arXiv, Feb. 2020. doi: 10.48550/arXiv.2002.05202.
- [45] Y. Shoshitaishvili *et al.*, “SOK: (State of) The Art of War: Offensive Techniques in Binary Analysis,” in *2016 IEEE Symposium on Security and Privacy (SP)*, San Jose, CA: IEEE, May 2016, pp. 138–157. doi: 10.1109/SP.2016.17.
- [46] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, “RoFormer: Enhanced Transformer with Rotary Position Embedding,” no. arXiv:2104.09864. arXiv, Nov. 2023. doi: 10.48550/arXiv.2104.09864.

Bibliography

- [47] H. Tan, Q. Luo, J. Li, and Y. Zhang, “LLM4Decompile: Decompiling Binary Code with Large Language Models,” in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024, pp. 3473–3487. doi: 10.18653/v1/2024.emnlp-main.203.
- [48] H. Touvron *et al.*, “Llama 2: Open Foundation and Fine-Tuned Chat Models,” no. arXiv:2307.09288. arXiv, July 2023. doi: 10.48550/arXiv.2307.09288.
- [49] H. Touvron *et al.*, “LLaMA: Open and Efficient Foundation Language Models,” no. arXiv:2302.13971. arXiv, Feb. 2023. doi: 10.48550/arXiv.2302.13971.
- [50] A. Vaswani *et al.*, “Attention Is All You Need,” no. arXiv:1706.03762. arXiv, Aug. 2023. doi: 10.48550/arXiv.1706.03762.
- [51] P. Wang *et al.*, “EasyEdit: An Easy-to-use Knowledge Editing Framework for Large Language Models.” arXiv, 2023. doi: 10.48550/ARXIV.2308.07269.
- [52] P. Wang *et al.*, “WISE: Rethinking the Knowledge Memory for Lifelong Model Editing of Large Language Models,” no. arXiv:2405.14768. arXiv, Dec. 2024. doi: 10.48550/arXiv.2405.14768.
- [53] J. Wei *et al.*, “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” no. arXiv:2201.11903. arXiv, Jan. 2023. doi: 10.48550/arXiv.2201.11903.
- [54] J. Wei *et al.*, “Emergent Abilities of Large Language Models,” no. arXiv:2206.07682. arXiv, Oct. 2022. doi: 10.48550/arXiv.2206.07682.
- [55] J. Wei *et al.*, “Finetuned Language Models Are Zero-Shot Learners,” no. arXiv:2109.01652. arXiv, Feb. 2022. doi: 10.48550/arXiv.2109.01652.

Bibliography

- [56] W. K. Wong *et al.*, “DecLLM: LLM-Augmented Recompilable Decompilation for Enabling Programmatic Use of Decompiled Code,” *Proceedings of the ACM on Software Engineering*, vol. 2, no. ISSTA, pp. 1841–1864, June 2025, doi: 10.1145/3728958.
- [57] W. K. Wong *et al.*, “Refining Decompiled C Code with Large Language Models,” no. arXiv:2310.06530. arXiv, Nov. 2023. doi: 10.48550/arXiv.2310.06530.
- [58] J. Xie, P. Cao, Y. Chen, K. Liu, and J. Zhao, “Revealing the Deceptiveness of Knowledge Editing: A Mechanistic Analysis of Superficial Editing,” in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Vienna, Austria: Association for Computational Linguistics, 2025, pp. 17756–17780. doi: 10.18653/v1/2025.acl-long.868.
- [59] Z. Xu *et al.*, “EasyEdit2: An Easy-to-use Steering Framework for Editing Large Language Models,” no. arXiv:2504.15133. arXiv, Sept. 2025. doi: 10.48550/arXiv.2504.15133.
- [60] J. Yoon, A. Gupta, and G. Anumanchipalli, “Is Bigger Edit Batch Size Always Better? – An Empirical Study on Model Editing with Llama-3,” no. arXiv:2405.00664. arXiv, May 2024. doi: 10.48550/arXiv.2405.00664.
- [61] N. Zhang *et al.*, “A Comprehensive Study of Knowledge Editing for Large Language Models,” no. arXiv:2401.01286. arXiv, Nov. 2024. doi: 10.48550/arXiv.2401.01286.
- [62] B. Zhang and R. Sennrich, “Root Mean Square Layer Normalization,” no. arXiv:1910.07467. arXiv, Oct. 2019. doi: 10.48550/arXiv.1910.07467.
- [63] C. Zheng *et al.*, “Can We Edit Factual Knowledge by In-Context Learning?,” no. arXiv:2305.12740. arXiv, May 2023. doi: 10.48550/arXiv.2305.12740.